Program 4

AIM:

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

Theory:

Prompt Enrichment using Word Embeddings

Word embeddings are a technique in Natural Language Processing (NLP) that represent words as numerical vectors based on their meaning and context. Words with similar meanings have similar vector representations. Popular models include Word2Vec and GloVe.

In this experiment, word embeddings are used to improve prompts given to a Generative AI model. First, a basic prompt is taken as input. Then, important words from the prompt are identified, and their semantically similar words are retrieved using a pre-trained embedding model.

These similar words are added to the original prompt to create an enriched prompt, which provides more context and detail. Both the original and enriched prompts are then given to a Generative AI model (such as FLAN-T5 or GPT-based models) to generate responses.

The outputs are compared in terms of:

- Detail (amount of information provided)
- Relevance (how well the response matches the topic)

It is observed that enriched prompts generally produce more detailed, informative, and contextually relevant responses, as the additional semantic information helps the model better understand the user's intent.

Program

Step 1: Install Libraries

```
!pip install transformers sentence-transformers gensim
```

Step 2: Load Word Embeddings (GloVe)

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import gensim.downloader as api

# Load embeddings
emb_model = api.load("glove-wiki-gigaword-100")
```

Step 3: Get Similar Words

```
def get_similar_words(word, top_n=2):
    try:
        return [w[0] for w in emb_model.most_similar(word, topn=top_n)]
    except:
        return []
```

Step 4: Enrich Prompt

```python
def enrich_prompt(prompt):
    words = prompt.split()
    enriched = []

    for word in words:
        enriched.extend(get_similar_words(word))

    return prompt + " " + " ".join(enriched)
```

Step 5: Load Free LLM (FLAN-T5)

```python
# Load FLAN-T5 model and tokenizer directly
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")
```

Step 6: Generate Outputs

```python
# Prompts
original_prompt = "Explain benefits of exercise"
enriched_prompt = enrich_prompt(original_prompt)

# Generate
def generate_response(prompt, max_length=100):
    inputs = tokenizer(prompt, return_tensors="pt").input_ids
    outputs = model.generate(inputs, max_length=max_length)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

original_output = generate_response(original_prompt)
enriched_output = generate_response(enriched_prompt)

print("Original:", original_output)
print("\nEnriched:", enriched_output)
```

```
Loading weights: 100% 282/282 [00:00<00:00, 267.15it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are p
The model 'T5ForConditionalGeneration' is not supported for text-generation. Supported models are ['PeftModelForCau
Both `max_new_tokens` (=256) and `max_length`(=100) seem to have been set. `max_new_tokens` will take precedence. F
Both `max_new_tokens` (=256) and `max_length`(=100) seem to have been set. `max_new_tokens` will take precedence. F
Original: Explain benefits of exercise include an inadequate energy or vitamin B8 found for these treatments... whi

Enriched: Explain benefits of exercise benefit costs the from exercises routinee or in how each person feels how ov
```

Step 7: Quantitative Comparison

1. **Length-Based Detail**

```python
def detail_score(text):
    return len(text.split())

print("Original Detail:", detail_score(original_output))
print("Enriched Detail:", detail_score(enriched_output))
```

```
Original Detail: 37
Enriched Detail: 59
```

2. **Relevance using Semantic Similarity**

```python
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

def relevance_score(prompt, output):
    emb1 = model.encode(prompt, convert_to_tensor=True)
    emb2 = model.encode(output, convert_to_tensor=True)
    return util.cos_sim(emb1, emb2).item()

print("Original Relevance:", relevance_score(original_prompt, original_output))
print("Enriched Relevance:", relevance_score(original_prompt, enriched_output))
```

```
Notes:
- UNEXPECTED    :can be ignored when loading fr
tokenizer_config.json: 100%
vocab.txt:  232k/? [00:00<00:00, 3.09MB/s]
tokenizer.json:  466k/? [00:00<00:00, 6.21MB/s]
special_tokens_map.json: 100%
config.json: 100%
Original Relevance: 0.571841835975647
Enriched Relevance: 0.5974737405776978
```

3. **Diversity (Unique Words)**

```python
def diversity_score(text):
    words = text.split()
    return len(set(words)) / len(words)

print("Original Diversity:", diversity_score(original_output))
print("Enriched Diversity:", diversity_score(enriched_output))
```

```
Original Diversity: 0.972972972972973
Enriched Diversity: 0.7966101694915254
```